



Université de  
Sherbrooke

**Université de Sherbrooke**

**SQL**

*Expressions spécialisées*

**SQL\_03b**

Christina KHNAISSER ([christina.khnaisser@usherbrooke.ca](mailto:christina.khnaisser@usherbrooke.ca))

Luc LAVOIE ([luc.lavoie@usherbrooke.ca](mailto:luc.lavoie@usherbrooke.ca))

*(les auteurs sont cités en ordre alphabétique nominal)*

*Scriptorum/Scriptorum/SQL\_03b-Expressions-specialisees, version 1.0.0.a, en date du 2026-01-13*

*— document de travail, ne pas citer —*

# Plan

|                             |    |
|-----------------------------|----|
| Introduction . . . . .      | 3  |
| 1. Présentation . . . . .   | 4  |
| 2. Agrégation . . . . .     | 6  |
| 3. Groupement . . . . .     | 9  |
| 4. Ordonnancement . . . . . | 11 |
| 5. Quantification . . . . . | 13 |
| 6. Contexte . . . . .       | 19 |
| Conclusion . . . . .        | 22 |
| Références. . . . .         | 23 |
| Définitions . . . . .       | 24 |

# Introduction

Le présent document a pour but de présenter une introduction du langage de définition de requêtes qu'on retrouve dans SQL.

# 1. Présentation

**Restriction**  
 $R \sigma \text{ cond}$

| A  | B  | C  |
|----|----|----|
| a1 | b1 | c1 |
| a2 | b2 | c2 |
| a3 | b3 | c3 |

**Projection**  
 $R \pi \{A, C\}$

| A  | B  | C  |
|----|----|----|
| a1 | b1 | c1 |
| a2 | b2 | c2 |
| a3 | b3 | c3 |

**Jointure naturelle**  
 $R \bowtie S$

| A  | B  | B  | C  | A  | B  | C  |
|----|----|----|----|----|----|----|
| a1 | b1 | b1 | c1 | a1 | b1 | c1 |
| a2 | b1 | b2 | c2 | a2 | b1 | c1 |
| a3 | b3 | b3 | c3 | a3 | b3 | c3 |
| a4 | b4 | b3 | c4 | a3 | b3 | c4 |

**Extension  
(augmentation)**  
 $R \alpha C:f$

| A  | B  | A  | B  | C        |
|----|----|----|----|----------|
| a1 | b1 | a1 | b1 | f(a1,b1) |
| a2 | b1 | a2 | b1 | f(a2,b1) |
| a3 | b3 | a3 | b3 | f(a3,b3) |
| a4 | b4 | a4 | b4 | f(a4,b4) |

**Renommage**  
 $R \rho A:C$

| A   | C   |
|-----|-----|
| ... | ... |

Une requête n'est pas autre chose qu'une expression SELECT.

```
requête ::=  
  [ Contexte ]  
  SELECT opMode { * | projection-extension }  
  FROM listeDeJointures  
  [ Restriction ]  
  [ Groupement ]  
  [ OpComplémentaire ]  
  [ Ordonnancement ]  
  [ Divers ]  
  
opMode ::=  
  [ DISTINCT | ALL ]
```

## 2. Agrégation

Les fonctions d'agrégations :

- AVG(col)
- COUNT(col)
- MAX(col)
- MIN(col)
- SUM(col)
- MEDIAN(col)
- STDDEV(col)
- STDDEV\_POP(col)
- STDDEV\_SAMP(col)
- VARIANCE(col)
- VAR\_POP(col)
- VAR\_SAMP(col)

Plusieurs autres disponibles avec PostgreSQL, dont :

- string\_agg
- xml\_agg
- json\_agg
- array\_agg
- range\_agg
- bit\_or, bit\_and

de très nombreuses fonctions statistiques...

## Exemple — Université

- Combien y-a-t'il d'activités offertes ?
- Quelle est la moyenne des résultats ?
- Calculer la moyenne, le minimum et le maximum des résultats des examens de l'activité IFT187.



### 3. Groupement

```
Groupement ::=  
  [ GROUP BY { expression ... , } ]  
  [ HAVING { condition ... , } ]
```

## Exemple — Université

- Quel est le nombre d'évaluations consigné par étudiant ?
- Quel est le nombre d'activités auxquelles un étudiant est (ou a été) inscrit ?
- Quelles sont les personnes étudiantes (ou a été) inscrites à plus que trois activités ?
- Quelle est l'activité n'ayant aucune évaluation ?
- Quel est la note totale par étudiant pour chacun de ces cours ?

## 4. Ordonnancement

Ordonnancement ::=

```
[ ORDER BY { expression [ ordre ] [ t_null ] ... , } ]  
[ LIMIT { nombre | ALL } ] [ OFFSET nombre ]
```

order ::=

```
ASC | DESC | USING opérateur
```

t\_null ::=

```
NULLS { FIRST | LAST }
```

## Exemple — Université

- Quelles sont les activités avec le plus d'inscriptions ?

## 5. Quantification

Un type de restriction dérivée de la logique du premier ordre.

```
restriction ::=  
  [ WHERE condition* ]
```

```
condition ::=  
  quantification_gen  
  | quantification_com  
  | expression
```

## 5.1. Quantification générale

```
quantification_gen ::=  
  [NOT] EXISTS (requête)
```

$\exists x. p(x)$

ensemble non vide

$0 <> (\text{SELECT count(*) FROM ensemble})$   
EXISTS (ensemble)

$\neg \exists x. p(x)$

ensemble vide

$0 = (\text{SELECT count(*) FROM ensemble})$   
NOT EXISTS (ensemble)

## Exemple — Université

- Quels sont les étudiants qui n'ont aucune évaluation à ce jour ?
- Quelles sont les activités n'ayant aucun étudiant à ce jour ?



## 5.2. Quantification de comparaison

```
quantification_com ::=  
    expression opComp [ANY | ALL] ensemble
```

```
opComp ::=  
    = | <> | < | <= | > | >=
```

```
ensemble ::=  
    ( requête ) | ( listeDeValeurs )
```

## Exemple — Université

- Quelles sont les personnes étudiantes dont toutes les notes sont supérieures ou égales à 60 ?

## 6. Contexte

```
Contexte ::=  
  WITH [ RECURSIVE ] { requêteCont ... , }  
  
requêteCont ::=  
  nomReqCont [ ( { nomCol ... , } ) ] AS ( opDefTable )  
  
opDefTable ::=  
  requête | valeurs | insertion | miseAJour | retrait
```

## Exemple — Université

- Quelles personnes étudiantes ne sont inscrites à aucune activité en 2013 ?
- Quelles sont les notes inférieures à la moyenne pour l'activité IFT187 ?  
Donner le matricule, le nom de la personne, la note et la différence avec la moyenne.

# UNIVERSITÉ

## Étudiant clé (matricule)

| matricule | nom     | adresse                                         |
|-----------|---------|-------------------------------------------------|
| 15113150  | Paul    | >Δ <sup>ς</sup> σ <sup>ς</sup> Δ <sup>ς</sup> b |
| 15112354  | Éliane  | Blanc-Sablon                                    |
| 15113870  | Mohamed | Tadoussac                                       |
| 15110132  | Sergeï  | Chandler                                        |

## Activité clé (sigle)

| sigle  | titre                             |
|--------|-----------------------------------|
| IFT159 | Analyse et programmation          |
| IFT187 | Éléments de bases de données      |
| IMN117 | Acquisition des médias numériques |
| IGE401 | Gestion de projets                |
| GMQ103 | Géopositionnement                 |

## TypeEvaluation clé(code)

| code | description      |
|------|------------------|
| IN   | Examen intra     |
| FI   | Examen final     |
| TP   | Travail pratique |
| PR   | Projet           |

## Résultat clé(matricule, te, activite, trimestre)

| matricule | TE | activité | trimestre | note |
|-----------|----|----------|-----------|------|
| 15113150  | TP | IFT187   | 20132     | 80   |
| 15112354  | FI | IFT187   | 20122     | 78   |
| 15113150  | TP | IFT159   | 20132     | 75   |
| 15112354  | FI | GMQ103   | 20122     | 85   |
| 15110132  | IN | IMN117   | 20123     | 90   |
| 15110132  | IN | IFT187   | 20133     | 45   |
| 15112354  | FI | IFT159   | 20123     | 52   |

Figure 1. Exemple — Université

# Conclusion

# Références

## [Elmasri2016]

Ramez ELMASRI et Shamkant B. NAVATHE;  
*Fundamentals of database systems*;  
7<sup>th</sup> Edition, Pearson, Hoboken (NJ, US), 2016;  
ISBN 978-0-13-397077-7.

## [Date2015a]

Chris J. DATE;  
*SQL and relational theory: how to write accurate SQL code*;  
O'Reilly Media, 2015;  
ISBN 978-1-4919-4117-1.

## PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

# Définitions

## SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]



Produit le 2026-02-03 16:19:30 UTC



**Université de Sherbrooke**